

ASSIGNMENT-10.4

NAME:E.NIKHIL

H.NO:2303A51847

B-13

Lab 9 – Code Review and Quality: Using AI to Improve Code Quality and Readability

Task 1: AI-Assisted Syntax and Code Quality Review

Scenario

You join a development team and are asked to review a junior developer's Python script that fails to run correctly due to basic coding mistakes. Before deployment, the code must be corrected and standardized.

Task Description

You are given a Python script containing:

- Syntax errors
- Indentation issues
- Incorrect variable names
- Faulty function calls

Use an AI tool (GitHub Copilot / Cursor AI) to:

- Identify all syntactic and structural errors
- Correct them systematically
- Generate an explanation of each fix made

Expected Outcome

- Fully corrected and executable Python code
- AI-generated explanation describing:
 - o Syntax fixes
 - o Naming corrections
 - o Structural improvements
- Clean, readable version of the script

Faulty code:

```
ass10.py > ...
1  def countEven(numbers):
2  count = 0
3  for num in Numbers
4  if num % 2 = 0:
5  count = count + 1
6  return count
7
8  nums = [1,2,3,4,5,6]
9  print("Even count:", count_even(nums))
10 |
```

Output:

```
PS C:\Users\shash\Downloads\AI AC> & C:\Users\shash\AppData\Local\Programs\Python\Python314\python.exe "c:/Users/shash/Downloads/AI AC/ass10.py"
File "c:\Users\shash\Downloads\AI AC\ass10.py", line 2
    count = 0
    ^^^^^
```

Corrected code:

```
ass10.py > ...
1  def count_even(numbers):
2      count = 0
3
4      for num in numbers:
5          if num % 2 == 0:
6              count += 1
7
8      return count
9
10
11  nums = [1, 2, 3, 4, 5, 6]
12  even_count = count_even(nums)
13
14  print("Even count:", even_count)
15
```

Output:

```
PS C:\Users\shash\Downloads\AI AC> & C:\Users\shash\AppData\Local\Programs\Python\Python314\python.exe "c:/Users/shash/Downloads/AI AC/ass10.py"
Even count: 3
```

AI Review:

Weaknesses:

- **Hardcoded Values:** Constants (e.g., file paths, hyperparameters) are hardcoded; use config files or arguments for flexibility.
- **Performance:** Data loading and processing could be optimized (e.g., avoid loading entire datasets into memory if large; consider Dask).
- **Testing:** No unit tests; add assertions or a test suite to validate functions.
- **Security/Edge Cases:** No handling for missing data or invalid inputs; add checks to prevent failures.
- **Code Quality:** Some long functions; break them into smaller ones. Missing type hints.

Suggestions:

- Refactor into classes for better encapsulation (e.g., a `ModelTrainer` class).
- Add logging with `logging` module instead of print statements.
- Implement cross-validation for model evaluation.
- Fix any linting issues (run `flake8 ass10.py` in terminal).

To propose fixes, share the code. For example, to add a test file:

```
import unittest
from ass10 import your_function # Adjust import

class TestAss10(unittest.TestCase):
    def test_example(self):
        # ...existing code...
        self.assertEqual(your_function(input), expected_output)
        # ...existing code...
```

Task 2: Performance-Oriented Code Review

Scenario

A data processing function works correctly but is inefficient and slows down the system when large datasets are used.

Task Description

You are provided with a function that identifies duplicate values in a list using inefficient nested loops.

Using AI-assisted code review:

- Analyze the logic for performance bottlenecks
- Refactor the code for better time complexity
- Preserve the correctness of the output

Ask the AI to explain:

- Why the original approach was inefficient
- How the optimized version improves performance

Expected Outcome

- Optimized duplicate-detection logic (e.g., using sets or hash-based structures)
- Improved time complexity
- AI explanation of performance improvement
- Clean, readable implementation

Inefficient code:

```
ass10.py > ...
1  def count_duplicates(numbers):
2      duplicate_count = 0
3
4      for i in range(len(numbers)):
5          for j in range(len(numbers)):
6              if i != j and numbers[i] == numbers[j]:
7                  duplicate_count += 1
8                  break
9
10     return duplicate_count
11
12
13     data = [10, 20, 30, 10, 40, 20, 50]
14     print("Duplicate Count:", count_duplicates(data))
15
```

Efficient code:

```
ass10.py > ...
1 from collections import Counter
2 def count_duplicates(numbers):
3     if not isinstance(numbers, list):
4         raise TypeError("Input must be a list")
5     counts = Counter(numbers)
6     return sum(count - 1 for count in counts.values() if count > 1)
7
8 # ...existing code...
9 data = [10, 20, 30, 10, 40, 20, 50]
10 print("Duplicate Count:", count_duplicates(data)) # Outputs: 4 (two 10s and two 20s)
11 # ...existing code...
```

Output:

```
PS C:\Users\shash\Downloads\AI AC> & C:\Users\shash\AppData\Local\Programs\Python\Python314\python.exe "c:/Users/shash/Downloads/AI AC/ass10.py"
Duplicate Count: 2
PS C:\Users\shash\Downloads\AI AC>
```

AI Review:

Weaknesses:

- **Efficiency:** $O(n^2)$ time complexity due to nested loops, inefficient for large lists (e.g., $n=1,000$ could be slow).
- **Accuracy:** Doesn't count total duplicate occurrences (e.g., for `[10,10]`, it returns 2, but there are 2 duplicates; here it returns 4, but there are 4 duplicate instances). If intent is total count, it's incorrect.
- **Redundancy:** Checks every pair, including unnecessary comparisons; breaks early but still wasteful.
- **Edge Cases:** No handling for empty lists, non-lists, or non-hashable types. Counts each "duplicate" position once, not frequencies.
- **Code Style:** Missing docstrings, type hints, and could be more Pythonic.

Suggestions:

- Use `collections.Counter` for $O(n)$ efficiency and accurate counting.
- Clarify intent: total duplicate instances vs. number of elements with duplicates.
- Add input validation.

Task 3: Readability and Maintainability Refactoring

Scenario

A working script exists in a project, but it is difficult to understand due to poor naming, formatting, and structure. The team wants it rewritten for long-term maintainability.

Task Description

You are given a poorly structured Python function with:

- Cryptic function names
- Poor indentation
- Unclear variable naming
- No documentation

Use AI-assisted review to:

- Refactor the code for clarity

- Apply PEP 8 formatting standards
- Improve naming conventions
- Add meaningful documentation

Expected Outcome

- Clean, well-structured code
- Descriptive function and variable names
- Proper indentation and formatting
- Docstrings explaining the function purpose
- AI explanation of readability improvements

Poorly structured code:

```
ass10.py > ...  
1  def cal(x,y):  
2      if x>5:  
3          b=y*0.2  
4      else:  
5          b=y*0.1  
6          t=y+b  
7      return t  
8  
9  res=cal(6,50000)  
10 print(res)  
11
```

Refactored code:

```
ass10.py > calculate_total_salary
1 def calculate_total_salary(years_of_service, base_salary):
2     if years_of_service > 5:
3         bonus = base_salary * 0.20
4     else:
5         bonus = base_salary * 0.10
6
7     total_salary = base_salary + bonus
8
9     return total_salary
0
1
2 total = calculate_total_salary(6, 50000)
3 print("Total Salary Including Bonus:", total)
4
```

Output:

```
● PS C:\Users\shash\Downloads\AI AC> & C:\Users\shash\AppData\Local\Programs\Python\Python314\python.exe "c:/Users/shash/Downloads/AI AC/ass10.py"
Total Salary Including Bonus: 60000.0
○ PS C:\Users\shash\Downloads\AI AC> 
```

AI Review:

Weaknesses:

- **Naming:** Function name `cal` is vague; should be descriptive (e.g., `calculate_bonus`, `apply_discount`).
- **Variable Names:** Single-letter variables (`x`, `y`, `b`, `t`) are unclear; use meaningful names.
- **Indentation:** Uses single spaces instead of 4-space indentation (PEP 8 violation).
- **Documentation:** No docstring explaining purpose, parameters, or logic.
- **Type Hints:** Missing type annotations for parameters and return value.
- **Error Handling:** No validation for invalid inputs (e.g., negative values, non-numeric types).
- **Magic Numbers:** Hardcoded thresholds (5, 0.2, 0.1) lack context; should be constants.
- **Code Clarity:** Unclear what 20% and 10% represent (bonus, tax, discount, etc.).

Suggestions:

- Rename function and variables descriptively.
- Add input validation.
- Use constants with meaningful names.
- Fix indentation to PEP 8 standard.
- Include a docstring explaining the business logic.

Task 4: Secure Coding and Reliability Review

Scenario

A backend function retrieves user data from a database but has security vulnerabilities and poor error handling, making it unsafe for production deployment.

Task Description

You are given a Python script that:

- Uses unsafe SQL query construction
- Has no input validation
- Lacks exception handling

Use AI tools to:

- Identify security vulnerabilities

- Refactor the code using safe coding practices
- Add proper exception handling
- Improve robustness and reliability

Expected Outcome

- Secure SQL queries using parameterized statements
- Input validation logic
- Try-except blocks for runtime safety
- AI-generated explanation of security improvements
- Production-ready code structure

Unsafe code:

```
ass10.py > ...
1  import sqlite3
2
3  def login(user, pwd):
4      conn = sqlite3.connect("users.db")
5      cur = conn.cursor()
6
7      sql = f"SELECT * FROM users WHERE username='{user}' AND password='{pwd}'"
8      cur.execute(sql)
9
10     if cur.fetchone():
11         return "Login Successful"
12     else:
13         return "Login Failed"
14
```

Secure code:

```
ass10.py > ...
1 import sqlite3
2 import hashlib
3 def hash_password(password):
4     return hashlib.sha256(password.encode()).hexdigest()
5 def login(username, password):
6     if not username or not password:
7         return "Invalid input."
8     try:
9         connection = sqlite3.connect("users.db")
10        cursor = connection.cursor()
11        hashed_pwd = hash_password(password)
12        query =
13        cursor.execute(query, (username, hashed_pwd))
14        user = cursor.fetchone()
15        if user:
16            return "Login Successful"
17        else:
18            return "Login Failed"
19    except sqlite3.Error as error:
20        print("Database error:", error)
21        return "System error. Try again later."
22    finally:
23        if 'connection' in locals():
24            connection.close()
25 if __name__ == "__main__":
26     user_input = input("Username: ")
27     password_input = input("Password: ")
28
29     print(login(user_input, password_input))
30
```

Output:

```
PS C:\Users\shash\Downloads\AI AC> & C:\Users\shash\AppData\Local\Programs\Python\Python314\python.exe "c:/Users/shash/Downloads/AI AC/ass10.py"
Username: Nikhil
Password: nikky@46
Database error: no such table: users
System error. Try again later.
PS C:\Users\shash\Downloads\AI AC>
```

AI Review:

CRITICAL SECURITY ISSUES:

1. **SQL Injection Vulnerability:** Uses f-string with direct string concatenation. Malicious input like `' OR '1'='1` or `admin'--` bypasses authentication entirely.

```
# Attacker input: user = "admin'--" , pwd = "any"
# Query becomes: SELECT * FROM users WHERE user = 'admin'--
# Result: Logs in as admin without knowing password
```

2. **Plain Text Passwords:** Passwords are stored and compared in plain text, exposing them if database is compromised. Should use hashing (e.g., bcrypt, argon2).
3. **No Input Validation:** Accepts any input without checking.

Other Issues:

- **Error Handling:** No try-except blocks; crashes if database missing or query fails.
- **Documentation:** No docstring or type hints.
- **Connection Management:** Should use context managers (`with` statement) for safer handling.
- **Code Style:** Uses abbreviated variable names (`conn`, `cur`, `sql`); less readable.
- **Timing Attacks:** Simple comparison doesn't protect against timing-based attacks.

Task 5: AI-Based Automated Code Review Report

Scenario

Your team uses AI tools to perform automated preliminary code reviews before human review, to improve code quality and consistency across projects.

Task Description

You are provided with a poorly written Python script.

Using AI-assisted review:

- Generate a structured code review report that evaluates:
 - o Code readability
 - o Naming conventions

- o Formatting and style consistency
- o Error handling
- o Documentation quality
- o Maintainability

The task is not just to fix the code, but to analyze and report on quality issues.

Expected Outcome

- AI-generated review report including:
 - o Identified quality issues
 - o Risk areas
 - o Code smell detection
 - o Improvement suggestions
- Optional improved version of the code
- Demonstration of AI as a code reviewer, not just a code generator

Poor Login Validation Script

```

ass10.py > ...
1  def c(u,p):
2      if u=="admin" and p=="1234":
3          return True
4      else:
5          return False
6
7  x=input("u:")
8  y=input("p:")
9  print(c(x,y))
10

```

Optional Improved Version

```

ass10.py > ...
1  def validate_credentials(username, password):
2      if not username or not password:
3          raise ValueError("Username and password cannot be empty.")
4
5      # Example credentials (should be stored securely in real systems)
6      valid_username = "admin"
7      valid_password = "1234"
8
9      return username == valid_username and password == valid_password
10
11
12 if __name__ == "__main__":
13     user_input = input("Enter username: ")
14     password_input = input("Enter password: ")
15
16     if validate_credentials(user_input, password_input):
17         print("Login Successful")
18     else:
19         print("Login Failed")
20

```

Output:

```
PS C:\Users\shash\Downloads\AI AC> & C:\Users\shash\AppData\Local\Programs\Python\Python314\python.exe "c:/Users/shash/Downloads/AI AC.py"
Enter username: nikhil
Enter password: nikky
Login Failed
PS C:\Users\shash\Downloads\AI AC> |
```

AI Review:

2. Code Style and Readability (PEP 8 Compliance)

- **Indentation:** Consistent (4 spaces), good.
- **Spacing:** Missing spaces around operators (e.g., `u=="admin"` should be `u == "admin"`). Add spaces for readability.
- **Line length:** All lines are short – good.
- **Blank lines:** Add a blank line after the function for separation.
- **Naming conventions:**
 - Function name `c` is unclear. Use `check_credentials` or `authenticate`.
 - Parameters `u` and `p` should be `username` and `password` (avoid abbreviations).
 - Variables `x` and `y` could be `username_input` and `password_input`.

3. Best Practices and Improvements

- **Docstring:** Add a docstring to describe the function.
- **Type hints:** Use `def c(username: str, password: str) -> bool:` for clarity.
- **Logic simplification:** The `else` block is redundant. You can simplify to:

```
return username == "admin" and password == "1234"
```

This reduces the function to one line.

- **Input handling:** `input()` returns strings, which is fine, but consider stripping whitespace: `username = input("u:").strip()`.
- **Security concerns** (Critical):
 - **Hardcoded credentials:** Never hardcode usernames/passwords in production code. This is a major security flaw – anyone with access to the code can see the credentials. Use environment variables, secure storage, or a proper auth system.
 - **No encryption:** Passwords should be hashed (e.g., with `bcrypt` or `hashlib`).
 - **No rate limiting:** Easy to brute-force.
 - **Plain text output:** Printing `True/False` exposes auth status – in a real app, handle failures gracefully without revealing details.
- **Testing:** Only manual testing shown. Add unit tests for different inputs.
- **Efficiency:** Simple and fast – no issues.